CPSC 2221 - Database Systems
Group Project - Implementation of a Relational Database

Project Title:	Tic Tac Toe Game Organizer
Project Milestone:	4

#	Student Name	Student ID	Email Address
1	Alex Verga	100251624	alex10293@gmail.com
2	Alberto Urquidi	100369508	urquidialberto5@gmail.com
3	Sukhmanpreet Singh	100424941	ssingh454@mylangara.ca
4	Tomoyoshi Sakai	100392239	tsakai02@mylangara.ca

By keying our names and student IDs in the above table, we certify that the work submitted with this cover page was performed solely by those whose names and student IDs are included above.

Also, we indicate that we are fully aware of the rules and consequences of plagiarism, as set forth by the Langara College.

1. Project Description

The TicTacToe Platform is a web application built using PHP, MySQL, HTML, CSS, and JavaScript. Players can play TicTacToe against an AI opponent, track their stats, view a live leaderboard, replay past matches move by move, and join tournaments. Administrators can manage achievements and oversee users. Tournament Managers can host tournaments.

Core features:

- Player authentication - sign up, log in, and session-based access control with role detection (Player, Manager, Admin)
- TicTacToe game vs AI - matches automatically saved to the database including all moves, result, and updated player stats
- Player profile - displays wins, losses, draws, ranking points, recent match history, and achievements
- Leaderboard - top 10 players by ranking points with current user highlighted even if outside top 10, plus nested aggregation results
- Tournament system - managers create tournaments, players join them through the Join Tournament page
- Achievement system - admins add, edit, and delete achievements; players earn them through gameplay
- Admin panel - oversee users with comment system, edit admin levels with projection query, division query for fully-reviewed users
- Real-time availability checking - username and email uniqueness verified instantly during sign up

The database schema follows 3NF normalization across 15 tables. All queries use prepared statements.

2. Installation Instructions

Step 1 - Install XAMPP

- Download XAMPP
- Install XAMPP with default settings
- Launch the XAMPP Control Panel
- Start both the Apache and MySQL modules

Step 2 - Copy Project Files

- Navigate to C:\xampp\htdocs
- Create a new folder named tictactoe
- Copy all submitted project files into the tictactoe folder (all files flat, no subfolders other than /assets)

Step 3 - Run setup

- Open browser and go to http://localhost/tictactoe/setup.php
- Enter your phpadmin password (leave blank if you have not changed password in config file)
- If successful, click 'Go to app' link to open website

Step 4 - Run the Application

- Click Sign Up to create a player account, or use Login > Staff Login for admin/manager access

Troubleshooting

- Connection error — make sure MySQL is running in the XAMPP Control Panel
- Setup.php not working
    - Open Command Prompt and navigate to the MySQL bin folder:

      ```
      cd C:\xampp\mysql\bin
      ```

    - Run the setup script:

      ```
      mysql -u root -p tictactoe < "C:\xampp\htdocs\tictactoe\create_tables.sql"
      ```

    - Enter your xampp admin password. Press Enter when prompted for no password (default XAMPP has no password)
    - Ensure your password is in the $password array in db.php
    - Open your browser and go to: http://localhost/tictactoe/home.html

- Access denied — press Enter for an empty password at the CMD prompt
- Pages not found — confirm the folder is named tictactoe inside htdocs

3. SQL Queries Used

Projection

File: Admin_handler.php

Admins select which fields to display from the Admin table using checkboxes.

```
-- Example: all fields selected
SELECT AdminID, Name, Email, Level FROM Admin ORDER BY AdminID;

-- Example: Name and Email only
SELECT Name, Email FROM Admin ORDER BY AdminID;
```

Selection

File: people_handler.php

Select the admin_comments before we update or add the admin_comments.

```
SELECT Admin_Comments
          FROM Oversees
          WHERE AdminID = ? and UserID = ?
```

File: achievement_handler.php

Select the AchievementID from the Achievement table before we update or add the achievement.

```
sELECT AchievementID FROM Achievement WHERE AchievementID = ?
```

Select the AchievementID from the Awarded table before we update or add the award.

```
SELECT AchievementID FROM Awarded WHERE AchievementID = ? and UserID = ?
```

Users can do selections by clicking the row of the table.

Join

File: get_games.php

Retrieves a player's recent match history. Determines Win/Loss/Draw result and opponent name based on which side the logged-in player was on. Uses LEFT JOIN for Tournament_Managed so AI games are included.

```
SELECT m.MatchID, m.Match_Date, m.Match_Time,
       COALESCE(t.Name, 'AI Match') AS TournamentName,
       CASE WHEN m.Winner_ID = ? THEN 'Win'
            WHEN m.Winner_ID IS NULL THEN 'Draw'
            ELSE 'Loss' END AS Result,
       CASE WHEN pi.Player1_ID = ? THEN g2.Username
            ELSE g1.Username END AS Opponent
```

```
FROM Match_Contained m
JOIN Plays_inMatch pi ON m.MatchID = pi.MatchID
LEFT JOIN Tournament_Managed t ON m.TournamentID = t.TournamentID
JOIN GameUser g1 ON pi.Player1_ID = g1.UserID
JOIN GameUser g2 ON pi.Player2_ID = g2.UserID
WHERE pi.Player1_ID = ? OR pi.Player2_ID = ?
ORDER BY m.Match_Date DESC, m.Match_Time DESC LIMIT 10;
```

File: get_matches.php

Searches match history by player username for the match replay viewer.

```
SELECT m.MatchID, m.Match_Date,
       g1.Username AS Player1, g2.Username AS Player2,
       gw.Username AS WinnerName
FROM Match_Contained m
JOIN Plays_inMatch pi ON m.MatchID = pi.MatchID
JOIN GameUser g1 ON pi.Player1_ID = g1.UserID
JOIN GameUser g2 ON pi.Player2_ID = g2.UserID
LEFT JOIN GameUser gw ON m.Winner_ID = gw.UserID
WHERE g1.Username = ? OR g2.Username = ?
ORDER BY m.Match_Date DESC, m.Match_Time DESC;
```

File: get_tournaments.php

Lists all tournaments with prize money, manager name, and whether the current user has already joined each one.

```
SELECT t.TournamentID, t.Name, t.StartDate, t.Difficulty,
       p.PrizeMoney, g.Username AS ManagerName,
       CASE WHEN j.PlayerID IS NOT NULL THEN '1' ELSE '0' END AS AlreadyJoined
FROM Tournament_Managed t
JOIN PrizeMoney p ON t.Difficulty = p.Difficulty
JOIN TournamentManager tm ON t.ManagerID = tm.ManagerID
JOIN GameUser g ON tm.ManagerID = g.UserID
LEFT JOIN Joins j ON j.TournamentID = t.TournamentID AND j.PlayerID = ?
ORDER BY t.StartDate ASC;
```

File: get_tournaments.php

It retrieves all users and their information, along with any associated admin oversight details if available. They can choose this join by clicking the Show admin comments button.

<join>

```
SELECT GameUser.UserID, GameUser.Username, GameUser.Email, Oversees.AdminID,
Oversees.Admin_Comments
            FROM GameUser
            LEFT JOIN Oversees ON Oversees.UserID = GameUser.UserID
            ORDER BY GameUser.UserID
```

<unjoin>

```
SELECT GameUser.UserID, GameUser.Username, GameUser.Email
            FROM GameUser
            ORDER BY GameUser.UserID
```

Division

File: people_handler.php

Finds all users who have been reviewed (have a non-empty comment) by every single admin in the system. They can choose this division by clicking the All Admins Reviewed check button.

```
SELECT u.UserID, u.Username, u.Email, o.AdminID, o.Admin_Comments
FROM GameUser u
LEFT JOIN Oversees o ON o.UserID = u.UserID
WHERE NOT EXISTS (
    SELECT * FROM Admin a
    WHERE NOT EXISTS (
        SELECT o2.AdminID FROM Oversees o2
        WHERE o2.UserID = u.UserID
          AND o2.AdminID = a.AdminID
          AND o2.Admin_Comments IS NOT NULL
          AND o2.Admin_Comments <> ''
    )
) ORDER BY u.UserID;
```

Aggregation 1 — Platform-wide statistics

File: total_stats.php

Calculates the average ranking points and total player count across all players. Displayed in the stats bar at the top of the leaderboard.

```
SELECT AVG(RankingPoints) AS OverallAvg FROM Player;
SELECT COUNT(*) AS TotalPlayers FROM Player;
```

Aggregation 2 — Player rank on leaderboard

File: display_players.php

Calculates the logged-in player's rank by counting how many players have more ranking points than they do. Used to show the current user's position even when outside the top 10.

```
SELECT COUNT(*) + 1 AS Rank FROM Player
WHERE RankingPoints > (
    SELECT RankingPoints FROM Player WHERE PlayerID = ?
);
```

Aggregation 3 —Top 10 leaderboard

File: display_players.php

Retrieves the top 10 players ordered by ranking points, joined with GameUser for username.

```
SELECT p.PlayerID, gu.Username, p.TotalWins,
       p.TotalLosses, p.TotalDraws, p.RankingPoints
FROM Player p
JOIN GameUser gu ON p.PlayerID = gu.UserID
ORDER BY p.RankingPoints DESC LIMIT 10;
```

Nested Aggregation with GROUP BY

File: total_stats.php

Finds tournament difficulty levels where the average ranking points of participating players exceeds the overall platform average. Groups players by difficulty level using joins, then filters with HAVING.

```
SELECT pm.Difficulty, AVG(p.RankingPoints) AS AvgRanking
FROM Player p
JOIN Joins j ON p.PlayerID = j.PlayerID
JOIN Tournament_Managed tm ON j.TournamentID = tm.TournamentID
JOIN PrizeMoney pm ON tm.Difficulty = pm.Difficulty
GROUP BY pm.Difficulty
HAVING AVG(p.RankingPoints) > (
    SELECT AVG(RankingPoints) FROM Player
)
ORDER BY pm.Difficulty;
```

Delete Operation (with Cascade)

File: people_handler.php

Deletes a GameUser record. Because all dependent tables (Player, Awarded, Oversees, Joins, Plays_inMatch via Match_Contained) are defined with ON DELETE CASCADE, all related records are automatically removed.

```
DELETE FROM GameUser WHERE UserID = ?;
```

File: Admin_handler.php

Deletes an Admin record. If the admin deletes their own account, the session is destroyed and they are redirected to the login page.

```
DELETE FROM Admin WHERE AdminID = ?;
```

Update Admin Level

File: Admin_handler.php

Updates the admin level to the new level provided.

```
UPDATE Admin SET Level = ? WHERE AdminID = ?;
```

Update Player Profile

File: update_profile.php

Builds the SET based on which fields the user changed (username, email, password). Only changed fields are included.

```
UPDATE GameUser SET Username = ?, Email = ?, Password = ?
WHERE UserID = ?;
```

Update Player Stats After Game

File: save_game.php

Increments the appropriate stat column and adds ranking points when a game ends. Win = +3 points, Draw = +1 point, Loss = +0 points.

```
UPDATE Player
SET TotalWins = TotalWins + 1, RankingPoints = RankingPoints + 3
WHERE PlayerID = ?;
```

Update Oversees Comment

File: people_handler.php

If the comment field is empty, the Oversees record is deleted. If the record doesn't exist yet, it is inserted. Otherwise it is updated.

```
UPDATE Oversees SET Admin_Comments = ?
WHERE AdminID = ? AND UserID = ?;
```

Update Tournaments Organized

File: create_tournament.php

Increments the manager's tournament count when they successfully create a new tournament.

```
UPDATE TournamentManager
SET Tournaments_Organized = Tournaments_Organized + 1
WHERE ManagerID = ?;
```

Save Game — Insert Match, Players, and Moves

File: save_game.php

Saves the match, along with its participants and moves into all the respective tables.

```
-- Auto-generate next MatchID
SELECT COALESCE(MAX(MatchID), 0) + 1 AS nextID FROM Match_Contained;


-- Insert match record (TournamentID NULL for AI games)
INSERT INTO Match_Contained (MatchID, Match_Date, Match_Time, Winner_ID,
TournamentID)
VALUES (?, CURDATE(), CURTIME(), ?, NULL);


-- Insert match participants
INSERT INTO Plays_inMatch (MatchID, Player1_ID, Player2_ID) VALUES (?, ?, ?);
```

```
-- Insert each board state as a move record
INSERT INTO Move_MadeIn (MatchID, Move_Number, Position) VALUES (?, ?, ?);
```

Match Replay

File: get_match_moves.php

Returns all board states for a given match in order, which the frontend replays move by move.

```
SELECT Position FROM Move_MadeIn
WHERE MatchID = ? ORDER BY Move_Number ASC;
```

Username and Email Availability

File: check_availability.php

Checks the username and email availability in the database.

```
SELECT UserID FROM GameUser WHERE Username = ?;
SELECT UserID FROM GameUser WHERE Email = ?;
```

Join Tournament

File: join_tournament.php

Checks whether the User has already joined to the tournament, and inserts the join record.

```
-- Check not already joined
SELECT 1 FROM Joins WHERE PlayerID = ? AND TournamentID = ?;

-- Insert join record
INSERT INTO Joins (PlayerID, TournamentID, JoinDate) VALUES (?, ?, CURDATE());
```

Create Tournament

File: create_tournament.php

Creates a new tournament into its respective table.

```
INSERT INTO Tournament_Managed
(TournamentID, Name, Difficulty, StartDate, ManagerID, SupervisorID)
VALUES (?, ?, ?, ?, ?, ?);
```

Achievement

File: achievement_handler.php

Adds, deletes, or edits an achievement, and adds the respective record to the Edits table. Also, Adds and deletes awards..

```
-- Add new achievement
INSERT INTO Achievement (AchievementID, Title, Description) VALUES (?, ?, ?);


-- Update existing achievement
UPDATE Achievement SET Title = ?, Description = ? WHERE AchievementID = ?;


-- Log the edit
INSERT INTO Edits (AdminID, AchievementID, EditDate) VALUES (?, ?,
CURRENT_DATE);


-- Delete achievement (cascades to Awarded and Edits)
DELETE FROM Achievement WHERE AchievementID = ?;


-- Add Award
INSERT INTO Awarded (AchievementID, UserID, AwardDate) VALUES (?, ?, NOW())


-- Delete award
DELETE FROM Awarded WHERE AchievementID = ? and UserID = ?
```

4. Complete File List

File	Type	Purpose
home.html + home.css	HTML/CSS	Landing page
play.html	HTML	Quick Match or Join Tournament
game.html + game.js + game.css	HTML/JS/CSS	TicTacToe game vs AI
matches.html	HTML	Match search and move-by-move replay viewer
leaderboard.html + leaderboard.css	HTML/CSS	Top 10 players + nested aggregation results
profile.html + profile.css	HTML/CSS	Player stats, match history, achievements
editprofile.html	HTML	Update username, email, password
login.html + login.css	HTML/CSS	Player login
createaccount.html + createaccount.css	HTML/CSS	Sign up (New Account + Upgrade Existing tabs)
join_tournament.html	HTML	Browse and join tournaments
selectRole.html	HTML	Staff role selection (Admin / Manager)
gameManagerLogin.html	HTML	Manager login
managerHome.html	HTML	Manager dashboard
managerProfile.html	HTML	Manager profile and achievements
managerLoginAtHome.html	HTML	Switch manager account
create_tournament.html	HTML	Create a new tournament
adminLogin.html + adminLoginAtHome.html	HTML	Admin login and switch admin
createadminaccount.html	HTML	Admin sign up
adminHome.html	HTML	Admin dashboard
adminProfile.html	HTML	Admin profile and supervised tournaments
editachievements.html + editachievements.css	HTML/CSS	Achievement CRUD
editAdmin.html	HTML	Edit admin levels

overseepeople.html	HTML	Oversee users
styles.css	CSS	Global styles, sidebar, buttons
login.php	PHP	Authenticate player/manager, return roles
adminLogin.php	PHP	Authenticate admin
logout.php	PHP	Destroy session
insert_user.php	PHP	Create GameUser + assign Player/Manager roles
insert_admin.php	PHP	Create Admin account
upgrade_account.php	PHP	Add Player or Manager role to existing account
save_game.php	PHP	Save match, moves, update player stats
update_profile.php	PHP	Update username, email, or password
create_tournament.php	PHP	Insert tournament, increment manager count
join_tournament.php	PHP	Check and insert Joins record
people_handler.php	PHP	Selection, division, update, delete for Oversees
Admin_handler.php	PHP	Projection, update, delete for Admin table
achievement_handler.php	PHP	Achievement table
display_players.php	PHP	Top 10 + current user rank
total_stats.php	PHP	Avg points, total players, nested aggregation
get_games.php	PHP	Player match history
get_matches.php	PHP	Match search by username for replay viewer
get_match_moves.php	PHP	Fetch move sequence for a match
get_tournaments.php	PHP	Tournament list
get_player.php	PHP	Player profile stats
get_achievements.php	PHP	Player achievements
get_admin.php	PHP	Admin profile data
get_manager.php	PHP	Manager profile data
get_supervise.php	PHP	Tournaments supervised by an admin

get_session.php	PHP	Player session state
get_adminSession.php	PHP	Admin session state
get_managerSession.php	PHP	Manager session state
get_user.php	PHP	Current user info for edit profile
check_availability.php	PHP	Check username/email uniqueness (players)
check_adminAvailability.php	PHP	Check name/email uniqueness (admins)
db.php	PHP	Shared database connection
create_tables.sql	SQL	Full database schema + sample data
assets/tttLogo.png + assets/logo.ico	Assets	Application logo and favicon